

Modulo 1A — Variabili, Condizioni, Loop Bash

Corso: Lab Amministrazione di Sistemi T

Prerequisito: Moduli 0A e 0B completati

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Cos'è uno script Bash

Finora hai usato i comandi uno alla volta nel terminale. Uno script Bash è semplicemente un file di testo che contiene una sequenza di comandi — la shell li esegue in ordine, uno dopo l'altro, come se li stessi digitando tu.

Questo è il punto di partenza dell'automazione: invece di ripetere 20 comandi ogni volta, scrivi lo script una volta e lo esegui quando serve.

Parte 1 — Il primo script

1.1 Creare la directory di lavoro

```
mkdir ~/lab1A && cd ~/lab1A
```

1.2 Creare ed eseguire il primo script

```
cat > saluto.sh << 'EOF'
#!/bin/bash
echo "Ciao, sono uno script Bash"
echo "La data di oggi è: $(date)"
echo "Sono l'utente: $(whoami)"
EOF
```

La prima riga `#!/bin/bash` si chiama **shebang**: dice al sistema operativo quale interprete usare per eseguire questo file. Senza di essa il sistema non saprebbe che è uno script Bash.

Rendi lo script eseguibile e lancialo:

```
chmod +x saluto.sh
./saluto.sh
```

Output atteso: tre righe con il messaggio, la data corrente e il nome utente.

Nota: `$(comando)` è la sintassi per la **command substitution** — il risultato del comando viene inserito inline nella stringa. `$(date)` esegue `date` e inserisce il suo output lì.

Parte 2 — Variabili

2.1 Assegnare e leggere variabili

```
cat > variabili.sh << 'EOF'
#!/bin/bash
```

```
NOME="vagrant"
ETA=25
SALUTO="Benvenuto"

echo "$SALUTO, $NOME!"
echo "Hai $ETA anni."
echo "Il nome ha ${#NOME} caratteri."
EOF
chmod +x variabili.sh
./variabili.sh
```

Output atteso:

```
Benvenuto, vagrant!
Hai 25 anni.
Il nome ha 7 caratteri.
```

Regole fondamentali sulle variabili: - Assegnazione: **VARIABILE=valore** — **senza spazi** intorno a = - Lettura: **\$VARIABILE** oppure **\${VARIABILE}** (le graffe sono necessarie in contesti ambigui) - **\${#VARIABILE}** restituisce la lunghezza della stringa - Per convenzione le variabili in Bash si scrivono in MAIUSCOLO

2.2 Variabili speciali

```
cat > argomenti.sh << 'EOF'
#!/bin/bash
echo "Nome dello script: $0"
echo "Primo argomento: $1"
echo "Secondo argomento: $2"
echo "Numero di argomenti: $# "
echo "Tutti gli argomenti: $@"
EOF
chmod +x argomenti.sh
./argomenti.sh ciao mondo
```

Output atteso:

```
Nome dello script: ./argomenti.sh
Primo argomento: ciao
Secondo argomento: mondo
Numero di argomenti: 2
Tutti gli argomenti: ciao mondo
```

Queste variabili speciali (**\$0**, **\$1**, **\$#**, **\$@**) sono fondamentali: permettono agli script di ricevere input dall'esterno senza hardcodare i valori.

2.3 Variabili d'ambiente

Alcune variabili esistono già nel sistema senza doverle definire:

```
echo $HOME
echo $USER
echo $PATH
echo $SHELL
```

`$PATH` è particolarmente importante: è la lista di directory in cui la shell cerca i programmi quando digiti un comando. Quando scrivi `ls`, la shell cerca `ls` in ogni directory elencata in `$PATH`.

Parte 3 — Condizioni

3.1 Struttura if/then/else

```
cat > controllo_numero.sh << 'EOF'
#!/bin/bash
NUMERO=10

if [ $NUMERO -gt 5 ]; then
    echo "$NUMERO è maggiore di 5"
else
    echo "$NUMERO non è maggiore di 5"
fi
EOF
chmod +x controllo_numero.sh
./controllo_numero.sh
```

Output atteso: 10 è maggiore di 5

La struttura è:

```
if [ condizione ]; then
    # comandi se vero
elif [ altra_condizione ]; then
    # comandi se altra condizione vera
else
    # comandi se tutto falso
fi
```

`fi` chiude il blocco `if` (è “if” al contrario — convenzione Bash).

3.2 Operatori di confronto

Per i **numeri** si usano flag letterali:

Operatore	Significato
<code>-eq</code>	uguale (equal)
<code>-ne</code>	diverso (not equal)
<code>-gt</code>	maggiore di (greater than)
<code>-lt</code>	minore di (less than)

Operatore	Significato
-ge	maggiore o uguale
-le	minore o uguale

Per le **stringhe**:

Operatore	Significato
=	uguale
!=	diverso
-z	stringa vuota (zero length)
-n	stringa non vuota

Per i **file**:

Operatore	Significato
-f file	esiste ed è un file regolare
-d dir	esiste ed è una directory
-e path	esiste (file o directory)
-r file	esiste ed è leggibile
-x file	esiste ed è eseguibile

3.3 Condizioni su file

```
cat > controllo_file.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -z "$FILE" ]; then
    echo "Errore: devi specificare un file come argomento"
    exit 1
fi

if [ -f "$FILE" ]; then
    echo "$FILE esiste ed è un file"
elif [ -d "$FILE" ]; then
    echo "$FILE esiste ed è una directory"
else
    echo "$FILE non esiste"
fi
EOF
chmod +x controllo_file.sh

./controllo_file.sh /etc/hostname
```

```
./controllo_file.sh /etc
./controllo_file.sh /file_inesistente
./controllo_file.sh
```

Output atteso (una riga per ogni invocazione):

```
/etc/hostname esiste ed è un file
/etc esiste ed è una directory
/file_inesistente non esiste
Errore: devi specificare un file come argomento
```

Nota `exit 1`: termina lo script con codice di uscita 1 (errore). Per convenzione `exit 0` indica successo, qualsiasi altro valore indica errore. I codici di uscita sono fondamentali negli script di automazione per sapere se un'operazione è andata a buon fine.

Parte 4 — Loop

4.1 Loop for

```
cat > loop_for.sh << 'EOF'
#!/bin/bash
for FRUTTO in mela pera banana arancia; do
    echo "Frutto: $FRUTTO"
done
EOF
chmod +x loop_for.sh
./loop_for.sh
```

Output atteso: quattro righe, una per ogni frutto.

Il `for` itera su una lista di elementi. La sintassi è:

```
for VARIABILE in lista_elementi; do
    # comandi
done
```

4.2 Loop for su file

```
cat > loop_file.sh << 'EOF'
#!/bin/bash
for FILE in /etc/*.conf; do
    echo "Trovato file di configurazione: $FILE"
done
EOF
chmod +x loop_file.sh
./loop_file.sh
```

Output atteso: una riga per ogni file `.conf` in `/etc`. Il `*` è un wildcard: la shell espande `*.conf` nella lista di tutti i file che corrispondono al pattern prima di passarla al loop.

4.3 Loop for numerico

```
cat > loop_numerico.sh << 'EOF'
#!/bin/bash
for I in $(seq 1 5); do
    echo "Iterazione numero $I"
done
EOF
chmod +x loop_numerico.sh
./loop_numerico.sh
```

Output atteso: cinque righe numerate da 1 a 5.

seq 1 5 genera la sequenza di numeri da 1 a 5. \$(seq 1 5) la inserisce come lista nel for.

4.4 Loop while

```
cat > loop_while.sh << 'EOF'
#!/bin/bash
CONTATORE=1

while [ $CONTATORE -le 5 ]; do
    echo "Contatore: $CONTATORE"
    CONTATORE=$((CONTATORE + 1))
done
EOF
chmod +x loop_while.sh
./loop_while.sh
```

Output atteso: cinque righe con il contatore che va da 1 a 5.

\$(espressione) è la sintassi per l'**aritmetica** in Bash. CONTATORE=\$((CONTATORE + 1)) incrementa la variabile di 1 ad ogni iterazione.

Parte 5 — Script completo

Metti insieme tutto quello che hai imparato in uno script che analizza i file di una directory:

```
cat > analizza_dir.sh << 'EOF'
#!/bin/bash
DIRECTORY=$1

if [ -z "$DIRECTORY" ]; then
    echo "Uso: $0 <directory>"
    exit 1
fi
```

```

if [ ! -d "$DIRECTORY" ]; then
    echo "Errore: $DIRECTORY non è una directory"
    exit 1
fi

echo "=== Analisi di: $DIRECTORY ==="

CONTATORE_FILE=0
CONTATORE_DIR=0

for ELEMENTO in "$DIRECTORY"/*; do
    if [ -f "$ELEMENTO" ]; then
        CONTATORE_FILE=$((CONTATORE_FILE + 1))
    elif [ -d "$ELEMENTO" ]; then
        CONTATORE_DIR=$((CONTATORE_DIR + 1))
    fi
done

echo "File trovati: $CONTATORE_FILE"
echo "Directory trovate: $CONTATORE_DIR"
echo "Totale elementi: $((CONTATORE_FILE + CONTATORE_DIR))"
EOF
chmod +x analizza_dir.sh

./analizza_dir.sh /etc
./analizza_dir.sh /var/log
./analizza_dir.sh

```

Leggi lo script riga per riga prima di eseguirlo — devi essere in grado di spiegare cosa fa ogni parte.

Esercizio finale

Scrivi uno script `cerca_file.sh` che: 1. Riceve come primo argomento un nome di file (o pattern) e come secondo argomento una directory 2. Se gli argomenti mancano, stampa un messaggio di utilizzo ed esce con errore 3. Cerca tutti i file che corrispondono al pattern nella directory specificata 4. Per ogni file trovato, stampa se è leggibile o no

Suggerimento: usa un loop `for` con il pattern come wildcard, e i test `-r` e `-f` per verificare i permessi.

Riepilogo

Sintassi	Funzione
<code>VAR=valore</code>	Assegna variabile (no spazi)

Sintassi	Funzione
<code>\$VAR / \${VAR}</code>	Legge variabile
<code>\$1, \$2, \$#, \$@</code>	Argomenti dello script
<code>\$(comando)</code>	Command substitution
<code>\$((espressione))</code>	Aritmetica
<code>if [cond]; then ... fi</code>	Condizionale
<code>for VAR in lista; do ... done</code>	Loop su lista
<code>while [cond]; do ... done</code>	Loop con condizione
<code>exit 0 / exit 1</code>	Termina con successo / errore
<code>-gt, -lt, -eq</code>	Confronto numerico
<code>-f, -d, -e, -r</code>	Test su file

Connessione con Security

Gli script Bash sono la base di quasi tutti i tool di automazione offensiva e difensiva: - Script di enumerazione che cicla su una lista di host con **for** - Script di monitoring che controllano in loop con **while** se un servizio è attivo - Script di risposta agli incidenti che verificano con **if** la presenza di file sospetti

Collegati

- [\[\[Appunti_modulo1A\]\]](#)
- [\[\[appunti_modulo1A_bash_scripting\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[troubleshooting_vm\]\]](#)
 · [\[\[metodo_studio_esami_pratici\]\]](#)